

Report Tecnico: Architettura Real-Time Dual-Arm Control

Ottimizzazione FSM tramite Funzioni Membro e Analisi di Complessità

Sviluppo Software di Controllo Robotico

Giugno 2026

Indice

1	Introduzione	2
2	Architettura e Struttura del Codice C++	2
2.1	Pattern FSM a Doppio Puntatore	2
2.2	Pre-allocazione e Azzeramento del Real-Time Jitter	2
3	Documentazione del Controllore	3
3.1	Flusso di Esecuzione della FSM	3
3.2	Guida per l'Aggiunta di un Nuovo Stato	3

1 Introduzione

Nei sistemi di controllo robotico operanti in ambiente deterministico e real-time (con frequenze operative comprese tra 200 Hz e 1 kHz), la minimizzazione della latenza computazionale e l'eliminazione del jitter di memoria sono requisiti fondamentali.

Questo documento analizza l'evoluzione dell'architettura software del modulo di controllo robotico denominato `DualArmControl`, sviluppato sul framework `mc_rtc`. L'architettura è stata ottimizzata con un pattern basato su un **Doppio Puntatore a Funzione Membro** a esecuzione deterministica pre-allocata.

2 Architettura e Struttura del Codice C++

La vecchia implementazione software si appoggiava su flag booleani locali per discriminare la prima esecuzione di uno stato dall'esecuzione ciclica. Il nuovo design introduce una separazione netta e strutturale tra la fase di ingresso *one-shot* in uno stato (*Entry*) e la successiva esecuzione ciclica (*Run*).

2.1 Pattern FSM a Doppio Puntatore

L'astrazione si basa sull'alias di tipo `StateMethod`, definito come puntatore a una funzione membro della classe di controllo:

```
1 using StateMethod = void (DualArmControl::*)();
2 StateMethod currentState;
```

Il meccanismo di transizione gestisce la sequenza logica di switch in modo sicuro:

```
1 void DualArmControl::transitionTo(StateMethod entryMethod,
2                                 StateMethod runMethod) {
3     if (!entryMethod || !runMethod) return;
4     stateTimer = 0.0;
5     (this->*entryMethod)();
6     currentState = runMethod;
7 }
```

Grazie a questa astrazione, il loop principale di controllo `run()` viene liberato da qualsiasi ramificazione condizionale o istruzione di salto indotto:

```
1 bool DualArmControl::run() {
2     (this->*currentState)();
3     return mc_control::MCController::run();
4 }
```

2.2 Pre-allocazione e Azzeramento del Real-Time Jitter

All'interno del costruttore e del metodo di `reset()`, tutti i task del solutore quadratico (QP) come `PostureTask`, `EndEffectorTask` e `ImpedanceTask` vengono allocati preventivamente nello spazio di memoria.

Durante l'esecuzione sincrona nei singoli stati della FSM, il controllore si limita ad attivare o disattivare i task allocati inserendoli o rimuovendoli dal solutore di ottimizzazione attraverso i metodi nativi del framework:

```
1 solver().addTask(leftEdgeTask);
2 solver().removeTask(leftEdgeTask);
```

Questo approccio previene allocazioni dinamiche a runtime.

3 Documentazione del Controllore

Questa sezione descrive le specifiche operative per l'utilizzo e l'estensione del modulo software.

3.1 Flusso di Esecuzione della FSM

La macchina a stati si sviluppa secondo tre macro-fasi sequenziali stabili:

1. **IDLE**: Stato di sicurezza iniziale. Vincola il robot nella configurazione iniziale per 3.0 secondi.
2. **INDEPENDENT**: Attiva i task operativi nello spazio cartesiano per gli end-effector destro e sinistro, guidando i bracci robotici verso coordinate operative indipendenti dello spazio di lavoro.
3. **COLLABORATIVE**: Disattiva i vincoli cinematici indipendenti e inizializza i task di controllo ad impedenza. Definisce un sistema di riferimento virtuale centrale interpolando la traiettoria per la manipolazione cooperativa di un corpo rigido.

3.2 Guida per l'Aggiunta di un Nuovo Stato

Per inserire un ulteriore stato logico all'interno della FSM (ad esempio uno stato denominato `stateEmergency`), è sufficiente seguire questa procedura in tre passaggi:

1. Definizione dell'Interfaccia nell'Header

Includere i prototipi dei metodi di Entry e di Run all'interno dell'area privata del file di testata:

```
1 void entryStateEmergency();  
2 void stateEmergency();
```

2. Sviluppo della Logica nel Sorgente

Implementare le funzioni isolando la logica di configurazione iniziale dai cicli di calcolo iterativi:

```
1 void DualArmControl::entryStateEmergency() {  
2     solver().removeTask(leftImpedanceTask);  
3     solver().removeTask(rightImpedanceTask);  
4 }  
5  
6 void DualArmControl::stateEmergency() {  
7     stateTimer += timeStep;  
8 }
```

3. Chiamata della Routine di Transizione

Eseguire lo switch dello stato chiamando il metodo di transizione preposto da un qualsiasi punto della FSM attiva utilizzando i relativi riferimenti di istanza:

```
1 this->transitionTo(&DualArmControl::entryStateEmergency,  
2                  &DualArmControl::stateEmergency);
```